

GNR Project 2 — Visual MCQ Solver

Detailed Project Report

Satwik Bhole (Roll No. 24B2498)

Divyaansh Narkhede (Roll No. 24B0981)

1 Project Overview

The project solves a closed-book vision-language task: given a PNG image of a deep-learning multiple-choice question (MCQ) with four options, predict the correct option as an integer in $\{1, 2, 3, 4\}$, or 5 to skip.

The competition rubric (from `sample_test_project_2/README.md`) is:

Outcome	Score
Correct (1–4)	+1
Incorrect (1–4)	−0.25
Skipped (5)	0
Hallucinated ($\notin 1\text{--}5$)	−1

Because a hallucinated answer is penalised four times more than a wrong one and infinitely more than a skip, the pipeline is designed to **default to 5 whenever anything is uncertain**.

Hard constraints

- At most 50 images per evaluation run.
- Wall-clock budget: 1 hour on a 48 GB NVIDIA L40s.
- **Internet is allowed only during environment setup. The inference script itself must not make any network calls.**
- Submission file: `submission.csv` with columns `image_name`, `option`.

2 Repository Layout

```
gnr-project/
|-- README.md           # Short user-facing overview
|-- REPORT.tex          # <- this file
|-- setup.bash          # One-shot env + weights bootstrap (needs internet)
|-- environment.yml     # Conda env spec (mirror of setup.bash)
|-- requirements.txt    # Pip-only spec (mirror of setup.bash)
|-- inference.py        # Offline entry point -> submission.csv
|-- generate_tests.py   # Builds 20 synthetic MCQ PNGs for self-evaluation
|-- notebook.ipynb     # Kaggle harness used during development
|-- .gitignore
|-- my_test_cases/      # 20 generated test images + ground-truth answers
|   |-- images/image_1.png ... image_20.png
|   |-- test.csv
|   |-- sample_submission.csv
```

```
| |-- submission.csv          # Last run's predictions
| '-- answers.csv            # Ground truth (exclude before submitting)
'-- sample_test_project_2/    # Reference test format provided by organisers
    |-- README.md
    |-- images/{image_1,image_2}.png
    |-- test.csv
    '-- sample_submission.csv
```

3 Approach

3.1 Model choice — Qwen2.5-VL-7B-Instruct

[Qwen/Qwen2.5-VL-7B-Instruct](#) is a vision–language model with strong document-understanding capability (text-in-image OCR plus multimodal reasoning). Selection drivers:

- **OCR quality.** The MCQs are typeset PDF-style images; Qwen2.5-VL is trained on synthetic and real document data and handles them without an external OCR step.
- **Reasoning.** The model can produce chain-of-thought derivations needed for shape and parameter-count questions (e.g. CNN output size).
- **Memory footprint.** Approximately 15 GB at fp16/bf16 – fits comfortably on the L40s (48 GB) and, with optional 4-bit **nf4** quantisation, on a single 16 GB T4.
- **Permissive license** and good Hugging Face / **transformers** integration.

3.2 Decoding strategy

- **Zero-shot prompt** with explicit chain-of-thought instructions (**inference.py**, lines 43–56).
- **Deterministic decoding:** `do_sample=False`, no temperature – reproducible across runs.
- `max_new_tokens=768` – enough for arithmetic derivations without bloating runtime.
- **Image preprocessing:** `min_pixels = 256*28*28`, `max_pixels = 1280*28*28`. The default Qwen2.5-VL `max_pixels` (~12.8 M) is huge and OOMs T4-class GPUs, while typeset MCQ pages are perfectly readable at ~1 MP (**inference.py**, lines 34–37).

3.3 Output parsing — defensive layered regex

The model is instructed to terminate every response with a single line:

```
FINAL: <A|B|C|D|SKIP>
```

`parse_answer()` (**inference.py**, lines 59–81) implements three fallback layers:

1. **Primary:** `FINAL\s*[:\-\]\s*\{0,2\}\(?(?([A-D]SKIP)?0,2|` — tolerates `FINAL: A`, `FINAL: **(A)**`, `FINAL - A`, etc.
2. **Heuristic fallbacks:** “answer is X”, “correct option is X”, “option X is correct”, “option: X”.
3. **Default:** if nothing matches, return 5 (skip) — never a hallucinated value.

This layering is the single most important risk-control mechanism: any failure mode (OCR garble, exception, malformed completion) collapses to a 0-point skip rather than a –1-point hallucination.

3.4 Per-image error containment

The inference loop (`inference.py`, lines 189–209) wraps each image in a `try / except / finally` block:

- Missing image file → write 5.
- Any runtime exception during generation → write 5.
- `torch.cuda.empty_cache()` after every image to keep VRAM stable across the full batch.
- `f.flush()` after every row so a mid-run crash still produces a partial valid `submission.csv`.

3.5 Quantisation toggle

`GNR_LOAD_IN_4BIT=1` enables `bitsandbytes nf4` 4-bit loading (~4–5 GB) for testing on smaller GPUs (`inference.py`, lines 96–106). On the target L40s this is unnecessary — leave it off for fp16/bf16 quality.

4 Offline-Inference Audit

Constraint: the script must not require internet at inference time.

Verdict: PASS. `inference.py` only makes two model-loading calls, both against a local path:

Call	Source
<code>Qwen2_5_VLForConditionalGeneration.from_pretrained(MODEL_DIR)</code>	local dir
<code>AutoProcessor.from_pretrained(MODEL_DIR, ...)</code>	local dir

`MODEL_DIR` defaults to `./model_cache/Qwen2.5-VL-7B-Instruct` — overridable via `GNR_MODEL_DIR`. When `transformers` is given an existing local directory, it loads weights directly without contacting the Hub. There are **no** `snapshot_download`, `requests`, `urllib`, `http`, or `hf_hub_download` calls inside `inference.py`; `generate_tests.py` and `setup.bash` contain the only network code, neither of which is invoked at inference time.

Recommended belt-and-braces (optional)

To make the offline guarantee explicit at runtime, export these before running inference:

```
export HF_HUB_OFFLINE=1
export TRANSFORMERS_OFFLINE=1
```

These make `transformers` raise immediately if any code path tries to reach the Hub — a useful sanity check for the no-internet evaluation environment.

Other potential network triggers — checked clean

- `qwen_vl_utils.process_vision_info` only processes local image paths.
- The processor’s chat template is loaded from the local model directory.
- No telemetry / `wandb` / `hf-transfer` is invoked.

5 Generated Test Set (generate_tests.py)

`generate_tests.py` produces a 20-question synthetic test set that mirrors the visual style of the organisers' `sample_test_project_2/` PNGs (matplotlib, serif body text, monospace code blocks). Topics are intentionally diverse and representative of the target distribution:

CNN output shape · Conv2d parameter count · CrossEntropyLoss API · BatchNorm inference behaviour · Inverted dropout · ResNet residual block · LSTM output shape · Adam vs. SGD-momentum · Vanishing-gradient activations · `nn.Embedding` output shape · Layer freezing · Xavier init · Softmax temperature · `ConvTranspose2d` output size · `DataLoader` behaviour · MultiheadAttention output shape · L1 vs. L2 regularisation · ReLU gradient · LR warm-up · `nn.Module.forward()` calling convention.

Outputs:

```
my_test_cases/  
|-- images/image_1.png ... image_20.png  
|-- test.csv                # input format expected by inference.py  
|-- sample_submission.csv   # all-5 baseline  
'-- answers.csv            # ground truth -- DO NOT submit
```

Reproduced result

Comparing the committed `my_test_cases/submission.csv` against `my_test_cases/answers.csv` row-by-row gives **20 / 20 match**.

The Kaggle-run log embedded in `notebook.ipynb` confirms [infer] Wrote `/kaggle/working/submission.csv` (762.3s total) — approximately 38s per image on Kaggle's 2×T4, well inside the 1-hour budget. On a single L40s the run will be considerably faster.

6 Step-by-Step Run Instructions

6.1 Prerequisites

- NVIDIA GPU (L40s 48 GB target; T4 16 GB works with `GNR_LOAD_IN_4BIT=1`).
- CUDA 12.6 driver/toolkit visible to the system.
- conda (Miniconda or Anaconda) on PATH.
- Internet access **for setup only**.
- Approximately 30 GB free disk for model weights and the conda env.

6.2 Setup (one command, internet required)

From the project directory:

```
bash setup.bash
```

This single script (`setup.bash`):

1. Locates a working conda install.
2. Creates conda env `gnr_project_env` with Python 3.11 (skips if it already exists).
3. Activates the env and upgrades `pip` / `wheel`.

4. Installs PyTorch 2.6.0 + torchvision 0.21.0 from the CUDA-12.6 wheel index.
5. Installs `transformers` ($\geq 4.49, < 4.55$), `accelerate`, `qwen-vl-utils[decord]==0.0.8`, `pillow`, `huggingface_hub`, `bitsandbytes`.
6. Optionally clones the repo if `inference.py` is missing (set `REPO_URL` first; can be skipped when running directly from the unpacked folder).
7. Pre-downloads **Qwen2.5-VL-7B-Instruct** into `./model_cache/Qwen2.5-VL-7B-Instruct/` via `huggingface_hub.snapshot_download`.

Warning. Before sharing or submitting, edit `REPO_URL` at the top of `setup.bash` (line 9) to point at your fork. Otherwise the clone step aborts.

Equivalent setup (manual, if not using `setup.bash`)

```
# 1. Conda env
conda create -n gnr_project_env python=3.11 -y
conda activate gnr_project_env

# 2. Deps
python -m pip install --upgrade pip wheel
pip install --index-url https://download.pytorch.org/whl/cu126 \
    "torch==2.6.0" "torchvision==0.21.0"
pip install \
    "transformers>=4.49.0,<4.55" \
    "accelerate>=1.3.0" \
    "qwen-vl-utils[decord]==0.0.8" \
    "pillow>=10.0.0" \
    "huggingface_hub>=0.27.0" \
    "bitsandbytes>=0.45.0"

# 3. Weights
python -c "from huggingface_hub import snapshot_download; \
snapshot_download(repo_id='Qwen/Qwen2.5-VL-7B-Instruct', \
local_dir='./model_cache/Qwen2.5-VL-7B-Instruct', max_workers=8)"
```

`environment.yml` and `requirements.txt` mirror the same dependency list and can be used as drop-in alternatives.

6.3 (Optional) Generate the local synthetic test set

```
conda activate gnr_project_env
pip install matplotlib # only needed for generation
python generate_tests.py
```

Produces `my_test_cases/images/*.png`, `test.csv`, `sample_submission.csv`, and `answers.csv` (do not submit `answers.csv`).

6.4 Inference (offline, no internet required)

```
conda activate gnr_project_env

# Optional: force-disable any HF network access as a safety net
export HF_HUB_OFFLINE=1
export TRANSFORMERS_OFFLINE=1
```

```
# Run on the organisers' sample directory
python inference.py --test_dir /absolute/path/to/sample_test_project_2

# ...or on the local synthetic set
python inference.py --test_dir "$(pwd)/my_test_cases"

# Add --debug to print full model responses (prompt iteration only)
python inference.py --test_dir "$(pwd)/my_test_cases" --debug
```

submission.csv is written to **the current working directory** (not the test directory). The expected `--test_dir` layout is exactly what the organisers describe:

```
<test_dir>/
|-- images/
|   |-- image_1.png
|   '-- ...
|-- test.csv          (column: image_name)
'-- sample_submission.csv (optional, ignored by inference.py)
```

6.5 Useful environment overrides

Variable	Default	Purpose
GNR_MODEL_DIR	./model_cache/Qwen2.5-VL-7B-Instruct	Point at a different local copy of the weights
GNR_MIN_PIXELS	256 · 28 · 28 (~200 K)	Lower bound for dynamic image resizing
GNR_MAX_PIXELS	1280 · 28 · 28 (~1 M)	Upper bound – raise if images look blurry, lower if you OOM
GNR_LOAD_IN_4BIT	0	1 enables bitsandbytes <code>nf4</code> quantisation for ≤ 16 GB GPUs
HF_HUB_OFFLINE	unset	Set to 1 to harden the offline guarantee
TRANSFORMERS_OFFLINE	unset	Set to 1 to harden the offline guarantee

6.6 Self-evaluation against ground truth

After `inference.py` finishes, verify accuracy on the synthetic set:

```
import csv
gt = {r['image_name']: r['option'] for r in
      csv.DictReader(open('my_test_cases/answers.csv'))}
pred = {r['image_name']: r['option'] for r in csv.DictReader(open('submission.csv'))}
correct = sum(gt[k] == pred[k] for k in gt)
print(f"{correct} / {len(gt)} correct")
```

The committed run reproduces **20 / 20**.

7 Submission Format Notes

The actual sample CSV (`sample_test_project_2/sample_submission.csv`) uses **two columns** – `image_name`, `option` – and `inference.py` matches that exactly. Note that `sample_test_project_2/README.md` shows a three-column header (`id,image_name,option`) in its prose, but the binding artefact is `sample_submission.csv`. The current implementation follows the CSV. If the grader strictly requires three columns, add an `id` column equal to `image_name` to the writer in `inference.py` (lines 188 and 208).

8 Potential Risks and Mitigations

Risk	Mitigation already in place
Model returns malformed answer line OOM on a single image	Layered regex + default-to-5 in <code>parse_answer</code> Per-image <code>try/except</code> writes 5; <code>torch.cuda.empty_cache()</code> between images
Crash mid-run	Per-row <code>f.flush()</code> keeps a valid partial submission
Image file missing / unreadable Hallucinated non-1–5 value	Caught explicitly, written as 5 Impossible – writer only emits values from $LETTER_TO_INT \cup \{5\}$
Accidental Hub call at inference	All loads use a local path; recommend <code>HF_HUB_OFFLINE=1</code> / <code>TRANSFORMERS_OFFLINE=1</code> for a hard guarantee
Submission column-mismatch with grader	Easy 1-line fix described above; current output matches the committed <code>sample_submission.csv</code>
Runtime budget	$\sim 38\text{s}/\text{image}$ on $2\times\text{T4} \Rightarrow \sim 12\text{ min}$ for 20 images; L40s is faster – well under the 1 h / 50 imgs cap

9 Citations

- Bai et al., *Qwen2.5-VL Technical Report*, 2025 — <https://github.com/QwenLM/Qwen2.5-VL>
- Hugging Face `transformers` — <https://github.com/huggingface/transformers>
- `qwen-vl-utils` — <https://pypi.org/project/qwen-vl-utils/>
- `bitsandbytes` (optional 4-bit loading) — <https://github.com/TimDettmers/bitsandbytes>